

AI-based Damage Detection for Power Grid Equipment with SaaS-oriented Design Midterm Report (With Figures)

Yujie Feng
Supervisor: Jin Hou

March 12, 2026

Abstract

This report is the midterm summary of the project “AI-based Damage Detection for Power Grid Equipment with SaaS-oriented Design.” Power equipment in operation faces typical defects such as bird nests, oil leaks, and local overheating; traditional manual inspection is costly, risky, and difficult to cover at scale. The project aims to build a scalable SaaS inspection platform: multi-modal data (UAV RGB, infrared, sensors, and historical records) serve as input; AI performs damage detection and identification; outputs include alerts, work orders, and visual analytics. By midterm, Module A (bird nest) has been completed as an end-to-end PoC, Module B (oil leak) has training and deployment in place, and Module C (infrared high-temperature) has been implemented and integrated in Flask (custom temperature range, OCR-based scale reading, watermark/legend suppression). The report summarizes the technical approach, completed work, open issues, and next steps, and is supplemented with training curves, validation figures, and deployment visualizations.

Contents

1	Overview of the Graduation Project (Thesis)	3
2	Overall Arrangement and Schedule of the Graduation Project (Thesis)	4
2.1	Phase Breakdown	4
2.2	Schedule Note	4
3	Completed Research Part of the Graduation Project (Thesis)	4
3.1	Module A: Bird Nest Detection PoC	4
3.1.1	Data and Augmentation	4
3.1.2	Model and Training Config	5
3.1.3	Validation Metrics and Training Curves	5
3.1.4	Inference Example	6
3.2	Module B: Oil Leak Detection	6

3.2.1	Data Preparation	6
3.2.2	Training Config	7
3.2.3	Validation Metrics and Training Results	7
3.2.4	Integration with Flask	7
3.2.5	Inference Example	8
3.3	SaaS Deployment and Flask Microservice	8
3.3.1	Features	8
3.3.2	API and Run Mode	9
3.3.3	Platform Role	9
3.4	Module C: Infrared High-Temperature Detection	10
4	Next Phase Work Plan	11
4.1	Track 1: SaaS Platform (weeks 4–10)	11
4.2	Track 2: New Modules (weeks 11–18)	12
4.3	Wrap-up and Defence (weeks 19–20)	12
5	Problems in the Graduation Project (Thesis) Work	12
5.1	Generalization	12
5.2	Technical Choices	12
5.3	Limitations and Risks	13

1 Overview of the Graduation Project (Thesis)

The objective of this graduation project is to research and implement an AI-based damage detection system for power grid equipment with a SaaS-oriented design, so as to automate the identification of typical defects—such as bird nests, oil leaks, and local overheating—and to deliver a scalable inspection platform that outputs alerts, work orders, and visual analytics. Damage detection on power equipment plays a critical role in operational safety and maintenance efficiency: manual inspection is costly, risky, and difficult to cover at scale, while incomplete or inconsistent inspection leaves room for undetected hazards. While rule-based or single-module solutions exist, limitations persist in unifying multiple damage types, integrating multi-modal data (UAV visible light, infrared, sensors, and historical records), and exposing detection capability as a shared service for multiple departments and downstream systems such as alert centres and work-order management.

This project focuses on a “vertical slice first, then horizontal scale” strategy: complete one module from data preparation through training to deployment, then extend to a multi-module registry and unified scheduling. Unlike ad-hoc scripts or isolated models, the proposed approach integrates instance segmentation (Mask R-CNN for bird nests, yielding both bounding boxes and masks for work-order linkage), object detection (YOLOv8 for oil leaks, balancing speed and box-level localization), and rule- and OCR-based infrared high-temperature detection into a single Flask microservice with a configurable model registry. Key design choices include deterministic data augmentation and pipeline-based preprocessing for training; unified use of mixed-precision training and learning-rate schedules across modules; and a REST API for batch upload and inference so that bird nest, oil leak, and infrared modules are all accessible through one interface. The SaaS-oriented design, with unified authentication, multi-tenant isolation, and model registry, provides a foundation for shared detection capability and integration with enterprise alert and work-order systems.

The project generally consists of the following: curating and annotating datasets in COCO or YOLO format for bird nest and oil leak modules, with scripts for train/validation split and augmentation within the training pipeline; designing and training a Mask R-CNN model for bird nest instance segmentation and a YOLOv8 model for oil leak detection, with validation metrics and deployment-ready checkpoints; implementing infrared high-temperature detection via OCR-based scale reading and pixel-to-temperature mapping, including watermark and legend suppression; and deploying all modules under `deploy/flask-service` with batch upload, on-demand inference, and result visualization. By midterm, Module A (bird nest) has been completed as an end-to-end PoC validating the full pipeline; Module B (oil leak) has been trained on the CSDN dataset and integrated with Flask (best validation mAP50 ≈ 0.39 , mAP50-95 ≈ 0.26); and Module C (infrared high-temperature) is implemented and integrated with user-defined alarm temperature range and OCR-based scale reading. The expected outcome is a demonstrable multi-module SaaS prototype that delivers model-as-a-service detection capability and provides a practical foundation for further expansion of inspection modules and multi-tenant deployment.

2 Overall Arrangement and Schedule of the Graduation Project (Thesis)

The overall arrangement and schedule align with the proposal, using a phased “two-track” plan (see the proposal schedule and Gantt chart for reference). The phases are as follows.

2.1 Phase Breakdown

- **Weeks 1–4 (completed):** Module A (bird nest detection) PoC: data preparation, Mask R-CNN training, engineering optimizations, and Flask microservice deployment; vertical slice from data to callable service validated.
- **Weeks 5–10:** Track 1 — SaaS platform build. Multi-tenant backend, RBAC, task orchestration, and AI model registry API; productize PoC training utilities.
- **Weeks 11–18:** Track 2 — New module extension. Module C (infrared) is already integrated; complete Module B (oil leak) full training and metrics; reuse platform data and training; consolidate all three modules in the registry and microservice catalog.
- **Weeks 19–20:** Final integration and defence. End-to-end testing for multi-module and multi-user scenarios, system optimization, final report writing, and defence preparation.

2.2 Schedule Note

The above matches the Work Plan from the proposal defence: complete a single-module vertical slice (Module A), then horizontal scale (Track 1) and new modules (Track 2), then integration and thesis. If the proposal contains a finer week-by-week or milestone breakdown, that document should be treated as the authoritative schedule. This staged approach ensures that each track delivers testable outcomes before moving to the next, reducing integration risk at the final stage.

3 Completed Research Part of the Graduation Project (Thesis)

3.1 Module A: Bird Nest Detection PoC

3.1.1 Data and Augmentation

A COCO-format bird nest dataset of 54 images (43 train, 11 validation) is used, with bounding boxes and instance segmentation polygons per image. Deterministic augmentation is used instead of generative methods (e.g. Stable Diffusion) because early trials showed that generative augmentation introduced physically implausible artifacts (e.g. rust on non-metallic parts, misaligned texture), hurting discrimination of real damage; deterministic augmentation preserves geometric and photometric consistency while still increasing diversity. Concretely: custom Random IoU Crop (scale 0.3–1.0) to force learning from partial and multi-scale objects; photometric jitter (brightness, contrast) to simulate field conditions; ResizeMax and PadSquare for fixed aspect ratio and input size.

The pipeline is implemented with `torchvision.transforms.v2` and is compatible with PyTorch 2.x and DataLoader.

3.1.2 Model and Training Config

- Model: Mask R-CNN (ResNet50 + FPN v2) for instance segmentation and precise damage localization.
- Input: 512×512 , batch size 4, 40 epochs (early stopping supported).
- Optimizer: AdamW, learning rate 5×10^{-4} , OneCycleLR; mixed precision (AMP); ~ 27 s per epoch; GPU utilization 70–85% after optimization.

3.1.3 Validation Metrics and Training Curves

On the COCO validation set: bbox $AP_{50} = 1.000$, segm $AP_{50} = 1.000$, segm $AP_{75} = 0.919$, $AR@[0.50:0.95] \approx 0.83$. Training loss decreased from ≈ 1.41 to 0.14 and validation loss from ≈ 0.65 to 0.19, levelling off around epoch 20 with no clear overfitting. PR curves stay near precision 1.0 at IoU=0.50 and drop slightly at IoU 0.75 and 0.90, indicating higher sensitivity to localization at stricter IoU; the IoU distribution is right-skewed, and score threshold 0.4–0.5 gives a good F1 trade-off; in deployment, confidence threshold can be tuned to balance missed detections and false positives.

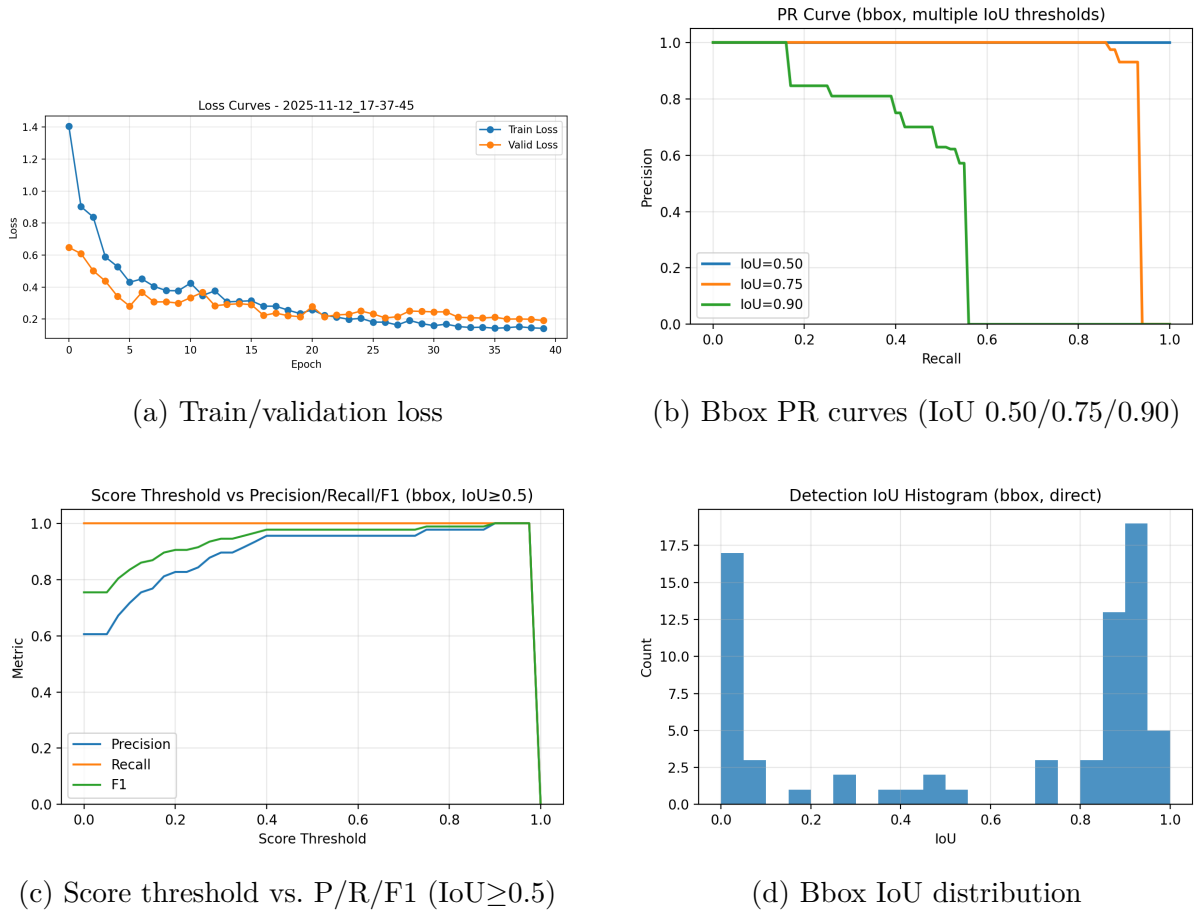


Figure 1: Bird nest: training loss and validation metrics; curves level off around epoch 20.

3.1.4 Inference Example

Predictions on validation and test scenes yield accurate bounding boxes and instance masks for bird nest hazard zones on transmission equipment. Figure 2 shows a typical detection result with box and mask overlay.

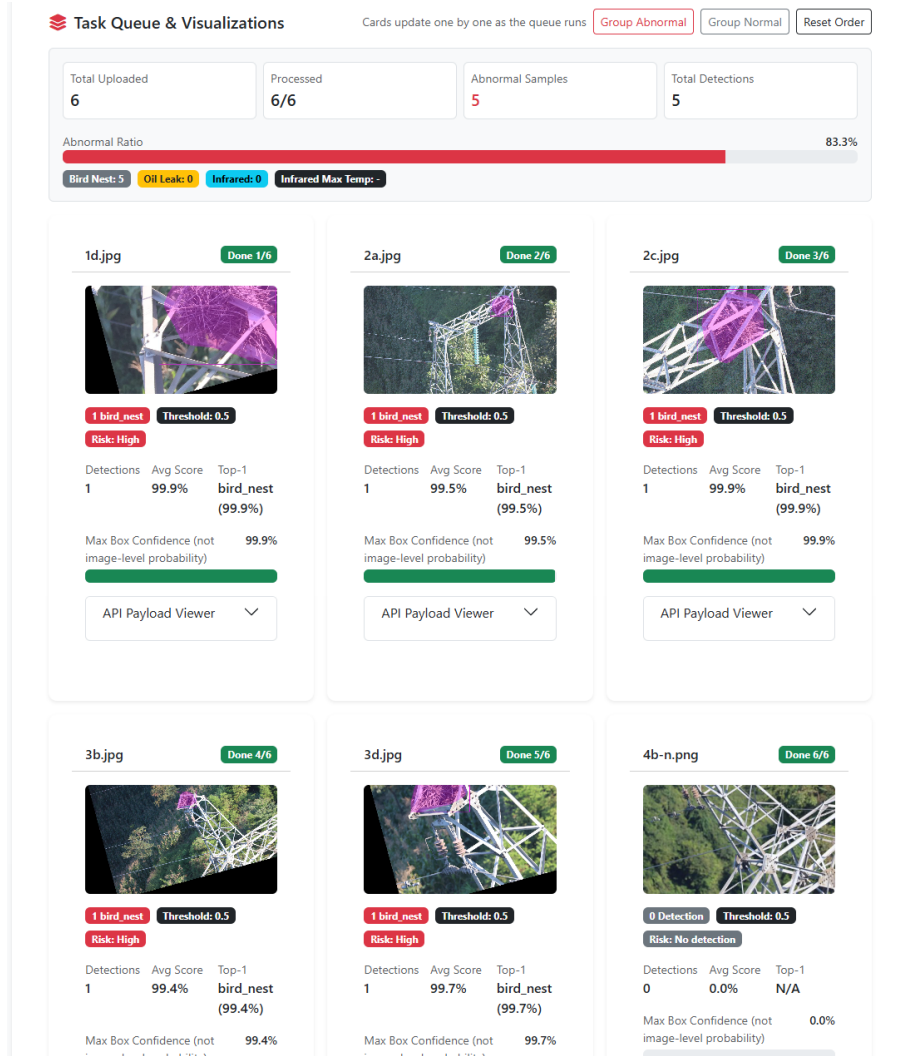


Figure 2: Bird nest detection result: bounding box and instance mask overlay.

3.2 Module B: Oil Leak Detection

3.2.1 Data Preparation

Oil leak data come from two sources: an original VOC-format dataset (~103 images with XML annotations) and YOLO-format data from sources such as CSDN. `prepare_dataset.py` supports three modes: VOC only, CSDN YOLO only, or merged (~333 images, ~8:2 train/val). The script converts VOC boxes to YOLO normalized format, uses a single class `oil_leak`, and writes a dataset YAML for YOLOv8. The training results reported here are from `runs/detect/train10-csdn`, which uses the **CSDN dataset** (`oil_leak_csdn.yaml`), i.e. YOLO-format data from the CSDN source only.

3.2.2 Training Config

YOLOv8s (`yolov8s.pt`) is used for single-class detection. Main hyperparameters for `train10-csdn`: 150 epochs, input 640, batch 8, AdamW, cosine LR, mosaic 0.3, mixup 0.05, RAM cache, patience 30, AMP enabled, fixed random seed 42. The training script is `src/oil_leak/train.py`; `--data`, `--epochs`, `--model`, etc. can be set from the command line.

3.2.3 Validation Metrics and Training Results

`train10-csdn` ran for 123 epochs; best validation metrics occur around epochs 93–94: **precision(B)** ≈ 0.83 , **recall(B)** ≈ 0.34 , **mAP50(B)** ≈ 0.39 , **mAP50-95(B)** ≈ 0.26 . Training losses (box/cls/dfl) decrease and stabilize over epochs. Figure 3 shows the validation PR curve, F1 curve, training results, and normalized confusion matrix. The best weights (`best.pt`) from this run can be copied to `checkpoints/oil_leak_yolov8.pt` for the Flask inference service.

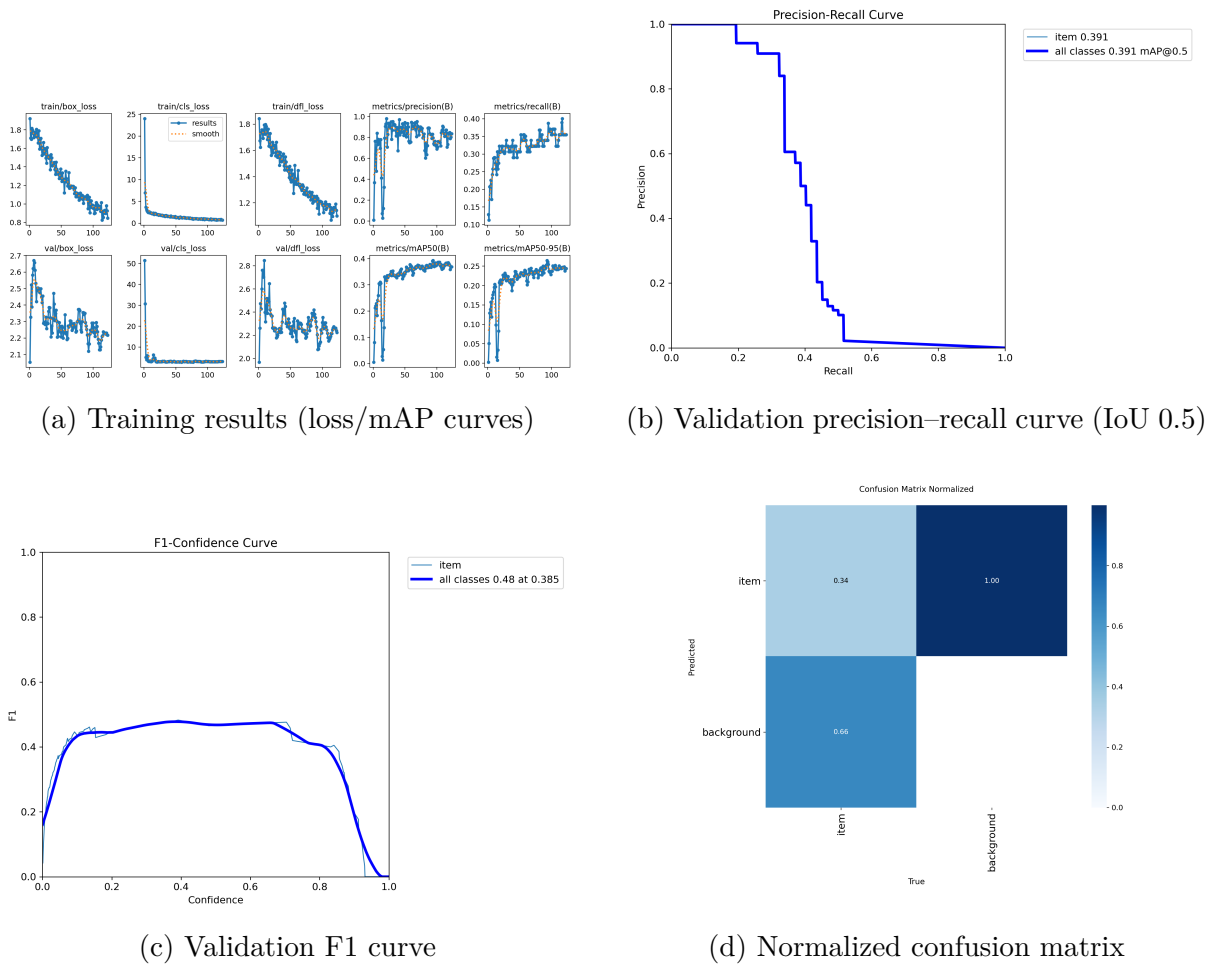


Figure 3: Oil leak (`train10-csdn`): training logs and validation metrics.

3.2.4 Integration with Flask

The Flask inference service supports the “oil leak” model: it loads YOLOv8 weights from the model registry `weights_path`, runs inference on uploaded images, and returns box coordinates, confidence, and overlay visualizations. If the registry path is missing

or the file is absent, the service falls back to `checkpoints/oil_leak_yolov8.pt` so that inference remains available before DB migration. The oil leak and bird nest modules share the same upload, inference, and download API; only the backend implementation is switched by model type, making it straightforward to add more detection modules in the same frontend.

3.2.5 Inference Example

Figure 4 illustrates localized detection bounding boxes on equipment images, confirming the system’s capability to identify oil leak regions.

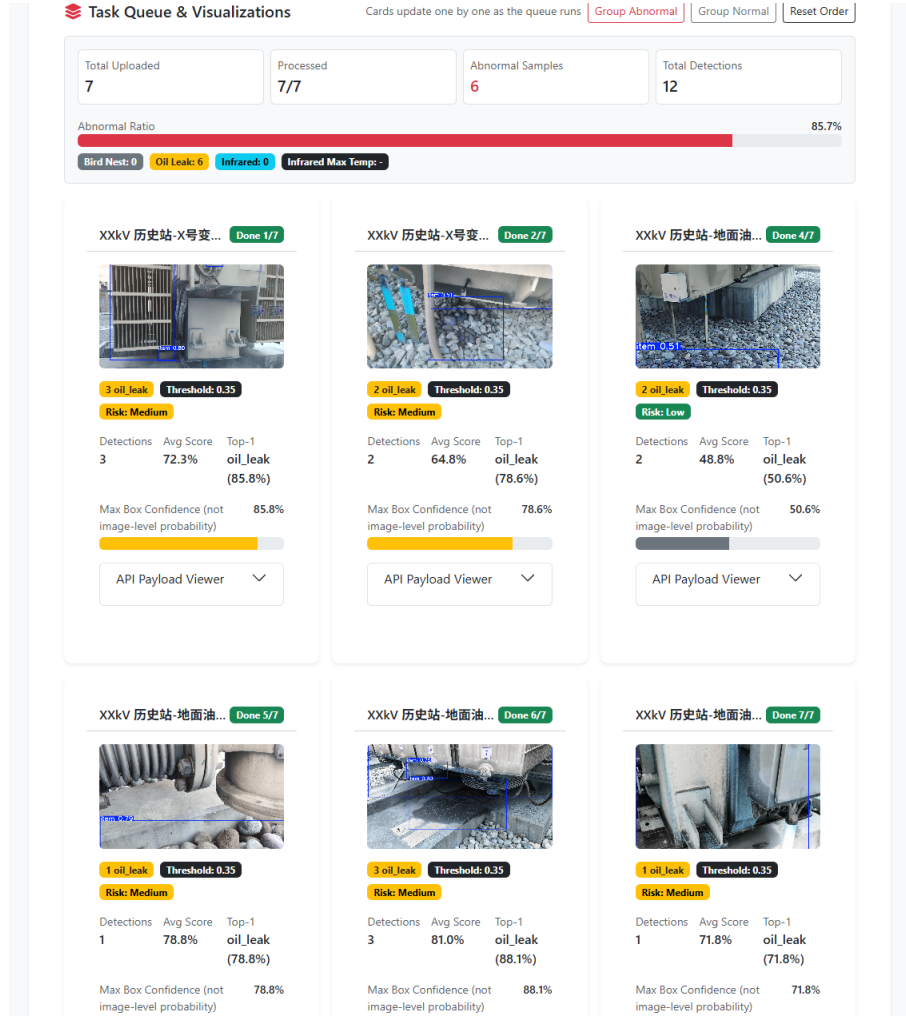


Figure 4: Oil leak detection result: identified leakage region on equipment.

3.3 SaaS Deployment and Flask Microservice

3.3.1 Features

Deployed under `deploy/flask-service`, the service provides:

- Batch image upload and on-demand inference: users select a registered model (e.g. “bird nest”, “oil leak”, or “infrared high-temperature”), upload one or more images; the server runs the corresponding logic (Mask R-CNN, YOLOv8, or infrared hybrid

detection) and returns JSON (boxes, confidence/temperature, class) and visualization images with boxes and masks.

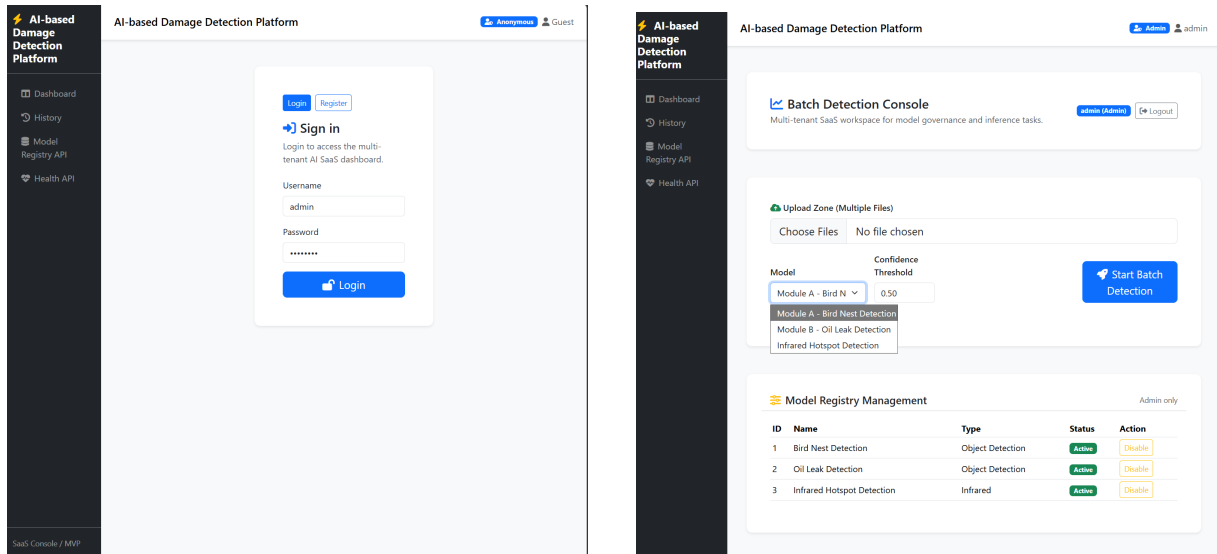
- Single and ZIP download: single result images or batch results as ZIP for offline review and archiving after field inspection.
- Statistics and dashboards: summary of historical inference tasks and detections for usage and alert distribution.
- Large-image handling and error handling: resize or tiling for big inputs to avoid OOM; exceptions are caught at the API layer with clear error messages for stability.

3.3.2 API and Run Mode

Request flow: user uploads images and selects model via the web UI → server resolves weight path by model type and runs inference → returns structured results and visualization URL. Development: `python run.py`, default `0.0.0.0:5000`; production: Gunicorn multi-worker (e.g. `gunicorn -w 4 wsgi:app`) for higher concurrency.

3.3.3 Platform Role

As a “model-as-a-service” deployment, bird nest, oil leak, and infrared high-temperature are all exposed through a single API; the configurable model registry and weight paths leave room for multi-tenant SaaS, RBAC, and task orchestration.



(a) Login interface

(b) Main dashboard and task flow

Figure 5: SaaS platform: authentication and main task flow UI.

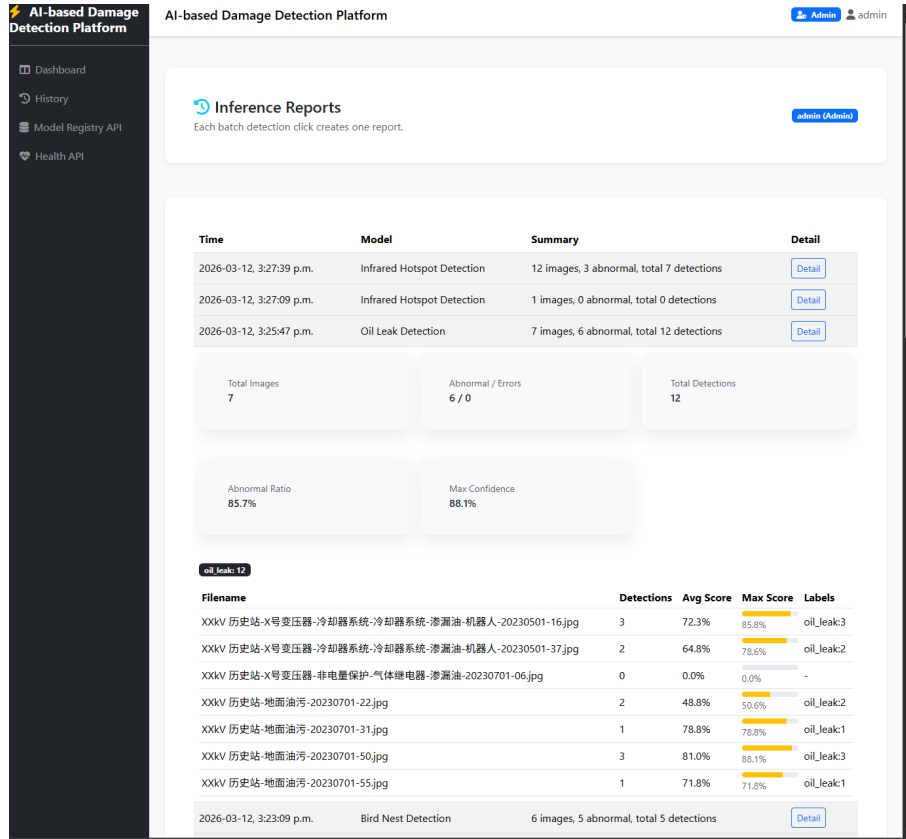


Figure 6: SaaS platform: history and statistical analytics for inference records and work orders.

3.4 Module C: Infrared High-Temperature Detection

Infrared high-temperature detection is implemented and integrated in deploy/flask-service, sharing the same upload, inference, and download API with bird nest and oil leak. Implementation is in app/services/model_service.py and app/routes/inference.py, using an OCR plus pixel-mapping hybrid (no separate deep-learning model training) for hotspot detection on infrared images.

Main features: (1) **User-defined alarm temperature range:** the front end or API passes an alarm temperature threshold (default 50°C); the service maps pixel intensity to temperature and filters regions above the threshold. (2) **OCR for high/low temperature scale:** EasyOCR is used to detect and recognize text on the infrared image; the maximum and minimum temperatures corresponding to the image scale are parsed from the legend or scale area and used to map pixel intensity to temperature, with heuristic correction for common OCR errors. (3) **Watermark and legend suppression:** a border watermark mask is built to detect and mask bright white watermark/logo regions near the image border so they are not treated as hotspots; OCR text-region masks exclude legend and scale digits to reduce false positives. This combination allows reliable hotspot detection without training a dedicated deep-learning model on infrared data. Outputs include hotspot boxes, per-hotspot temperature, global max read temperature, max hotspot temperature, and an annotated visualization.

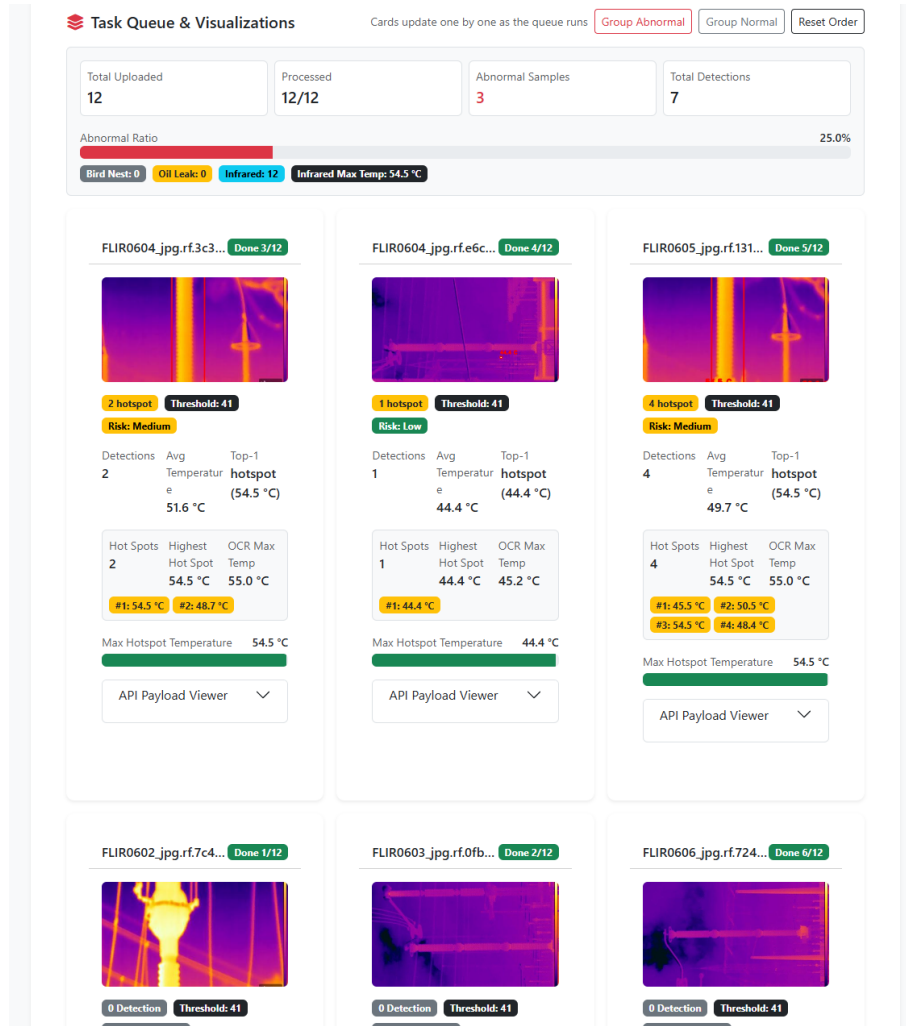


Figure 7: Infrared high-temperature detection: thermal mapping and hotspot bounding above user-defined threshold.

4 Next Phase Work Plan

4.1 Track 1: SaaS Platform (weeks 4–10)

- Multi-tenant backend, RBAC, and task orchestration: data models and APIs for users/roles/permissions; task queue (e.g. Celery or in-memory) and task-status queries so multiple users can submit detection tasks and view results concurrently.
- AI model registry API: unified registration, query, enable/disable, and weight-path management for models so the frontend and scheduler can select by name or type and to allow future versioning and A/B testing.
- Productize PoC training utilities: make augmentation config, training logs, and checkpoint management from the current scripts configurable and reusable for the oil leak and infrared modules to avoid duplication.

4.2 Track 2: New Modules (weeks 11–18)

- Complete Module B (oil leak) experiments and consolidate Module C (infrared): full training and validation metrics for oil leak on the merged dataset and registry update; infrared is already deployed—subsequent work can focus on robustness and multi-device infrared image adaptation.
- Reuse platform data and training: new modules should reuse `prepare_dataset`, training scripts, and deployment pipeline, only changing data paths and model type to reduce maintenance.
- Register new modules: add new model entries in the database and frontend so task routing and inference logic correctly recognize and invoke them.

4.3 Wrap-up and Defence (weeks 19–20)

Final integration, system testing, and report writing: end-to-end tests for multi-module and multi-user scenarios; consolidate experiment data and screenshots; complete the final report and defence materials. Expected deliverables: a demonstrable three-module (bird nest / oil leak / infrared) SaaS prototype and a written report and defence presentation that meet the requirements.

5 Problems in the Graduation Project (Thesis) Work

5.1 Generalization

Current bird nest validation gives $AP_{50}=1.0$ and $IoU \approx 1.0$ on a same-distribution validation set. In real complex conditions (haze, backlight, new tower types, different seasons and regions), performance may drop. Data augmentation, negative and hard-sample design, and multi-scene evaluation are therefore platform-wide needs: the plan is to add cross-scene data or hold out field data not used in training for generalization tests, and to apply the same care to negative and hard samples in the oil leak and infrared modules, avoiding overfitting to small, same-distribution data at the cost of real-world robustness.

5.2 Technical Choices

- **SaaS backend:** Whether to prioritize a feature-complete prototype (e.g. simple RBAC, task queue, multi-tenant isolation) or invest in production-grade auth (OAuth2, audit logs) from day one depends on time and defence goals; the current preference is to deliver a demonstrable multi-model, multi-task prototype first, then add auth and auditing if time allows.
- **Oil leak module:** YOLOv8 box-level detection is sufficient for “presence and approximate location”; if the business later requires precise oil-boundary segmentation (e.g. area statistics, trend comparison), semantic segmentation (e.g. U-Net) or combination with instance segmentation can be considered, with pixel-level annotations.
- **Infrared module:** Infrared high-temperature detection is already integrated as a standalone module with custom alarm temperature and OCR scale reading; follow-

up may consider fusion with RGB for multi-modal analysis (e.g. RGB + heatmap), depending on field data and operator workflow.

5.3 Limitations and Risks

Dataset size is still modest (54 bird nest images, ~333 merged oil leak); annotation quality and consistency have a strong impact on model performance. On the deployment side, Flask with multi-process can handle moderate concurrency; for higher load or distributed inference, a queue and worker pool would be needed. Large UAV images may require resize or tiling to avoid OOM during batch inference. The trade-off between implementing full enterprise SaaS features (e.g. detailed authentication and audit logs) and further improving the computer vision models remains an active consideration for the final project phase. These limitations will be addressed in later phases through data expansion and architecture iteration.